

TripMate AI: Intelligent Group Activity & Travel Planning Platform

Powered by a Unified DeepSeek-V4 AI Agent Architecture

Technical Project Proposal for Academic Supervisor

Project Team (6 Members): Ta Quang Huy Ha Dang Huy Pham Dinh Anh Duong Nguyen Minh Duc
Nguyen Tung Duong Dinh Tien Luan

1. Introduction, Scope & Product Overview

1.1. The Problem: Why Group Planning is Painful

Planning an outing or trip with friends—whether a weekend cafe hangout or a 4-day vacation—is often stressful and time-consuming:

- **Too many open tabs:** Finding good places means switching between Google Maps, food reviews, Agoda, Booking.com, and blogs for hours.
- **Different personal tastes:** Everyone wants something different (for example: a quiet cafe vs. a busy pub, cheap street food vs. a nice restaurant). Discussions often get stuck because nobody can agree.
- **Lost in chat:** Recommendations, links, opening hours, and polls quickly get buried under hundreds of messages in Zalo or Messenger.
- **Tired organizer:** Usually, one person ends up doing all the hard work of building the schedule, checking opening hours, and finding routes—and gets blamed if plans go wrong.

1.2. Target Users & Scope: Keeping It Practical

To keep the project clear, realistic, and focused on software engineering, **TripMate AI** sets clear boundaries:

- **Target Users:** Groups of friends, university students, young families, and coworkers planning leisure hangouts, dining, or trips together.
- **Scope (No Visa Hassles):** We focus strictly on trips where users **already have the right to travel** (domestic trips in Vietnam, visa-free trips in ASEAN countries, or travelers who already hold valid tourist visas).

1.3. The Solution: Two-Stage Consensus Planning Flow

TripMate AI makes planning simple through a shared room with a helpful AI travel guide:

1. **Chat Ideas & Save to Database:** Friends chat naturally about dates, approximate budget, and favorite vibes. All messages and preferences are saved directly into PostgreSQL.
2. **Macro Plan Overview:** The AI first suggests a high-level roadmap (number of days, main areas, and daily themes).
3. **Lock Macro Plan (Macro Consensus):** The group discusses and agrees on the big picture before picking specific places.
4. **Micro-Spot Curation:** Once the big picture is agreed, the AI fills each day with real places (morning coffee, lunch, afternoon activity, dinner) located close to each other via live APIs (Google Places, Agoda, Klook).
5. **Granular Voting & Swaps:** Members vote thumbs UP/DOWN on individual stops. Anyone can swap a disliked stop with a pre-checked alternative in one click.

6. **Final Plan Lock (Final Consensus):** The group locks the completed plan. It is saved in the database, exportable to a calendar file (.ics) and PDF, and remains editable live on the go.

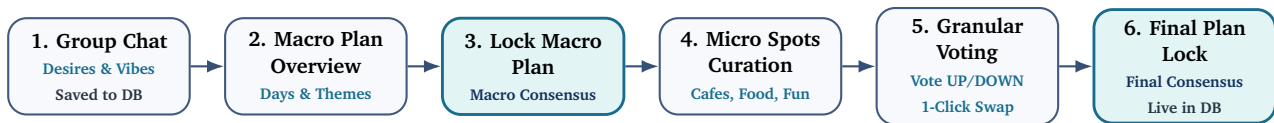


Figure 1: Two-stage planning flow: gather desires → macro plan overview → lock macro → curate micro spots → granular vote → final plan lock

2. Platform Features & User Experience

2.1. Two Flexible Planning Scopes

- **City Hangouts & Day Outings:** Quick plans for spontaneous weekend activities (for example: “*specialty cafe* → *board game lounge* → *hotpot dinner*”). Focuses on places open right now with short travel distances.
- **Multi-Day Vacations:** Full travel itineraries (for example: “*4 days in Da Nang & Hoi An*”). Combines hotel recommendations and travel routes while **planning every single day in detail** (breakfast spots, morning sights, scenic cafes, dinner, and evening entertainment).

2.2. Shared Group Room & Saved User Data

- **Simple Link Access:** Friends join instantly through a shareable link. Roles include *Host* (can lock the plan and manage members), *Member* (can chat, vote on stops, swap places), and *Viewer* (read-only).
- **Persistent Group Chat:** Friends talk naturally in chat; user messages, individual food preferences (vegetarian, seafood allergies), and budget limits are **saved directly into PostgreSQL** so the AI never forgets them.
- **Event Categories:** TRAVEL, DINING, HANGOUT, ENTERTAINMENT, SIGHTSEEING, and CUSTOM.

2.3. Interactive Timeline & Live Map

- **Interactive Cards:** The schedule is displayed as an interactive timeline. Each card shows real photos, verified open hours, estimated visit duration, price range, and direct booking links.
- **Mapbox Map Integration:** Displays clear location pins, travel routes, and transit times. Clicking any stop on the schedule centers the map on that place.
- **Drag-and-Drop Reordering:** Members can drag and drop stops to reorder or remove them, and travel times update automatically.

2.4. Two-Tier Group Consensus: Macro Lock & Micro Lock

The group always makes decisions together in two simple stages:

- **Stage 1 (Macro Approval):** The group votes on the overall plan (number of days, cities, daily themes). Once everyone is happy, the Host locks the macro roadmap.
- **Stage 2 (Micro Stop Voting):** The AI fills in specific venues. Members vote UP/DOWN on individual stops. Any downvoted place immediately offers 2 backup options with a 1-click swap button.
- **Final Consensus & Live Edits:** Once all stops are approved, the Host clicks **Confirm Plan**. Confirming does not lock the database permanently; the plan stays editable and can be updated anytime during the trip.

3. The AI Agent: Architecture, Workflow & Storage

3.1. Single Unified AI Agent: DeepSeek-V4 with Tool Calling

Instead of managing multiple disconnected bots, TripMate AI uses a **single, unified AI Agent powered by DeepSeek-V4**:

- **Unified Intelligence:** One cohesive agent handles the entire planning lifecycle. It reads member messages, understands group preferences, creates the high-level roadmap, and calls external tools to find venues.
- **LangGraph State Management:** Built with Python's **LangGraph**, the agent tracks the conversation state step by step, ensuring context is never lost across browser refreshes.
- **Tool Calling & Anti-Zigzag Routing:** The agent uses function calling to query external travel APIs and clusters venues within a 2–3 km radius per half-day block so groups do not waste time traveling back and forth across town.

3.2. Agent Architecture: User Data, System Interfacing & APIs

The AI Agent serves as the central intelligent brain connecting the user interface, backend services, database storage, and external travel APIs:

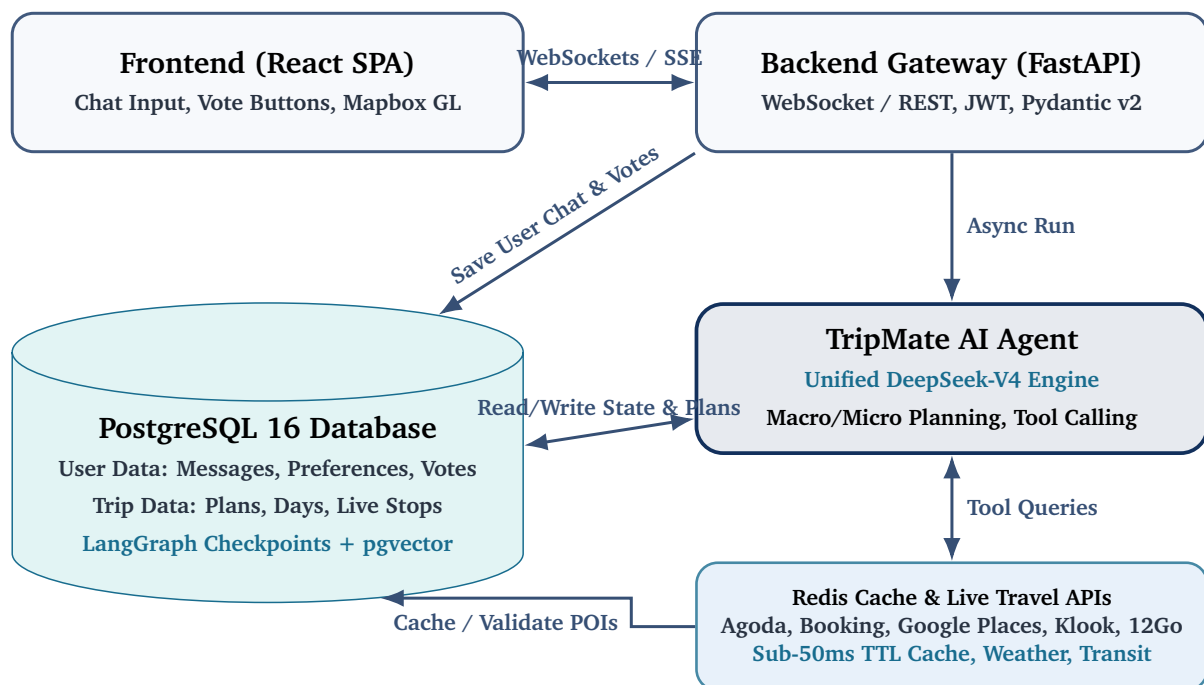


Figure 2: TripMate AI Agent architecture: user data persistence, system interfacing, and live API access

1. **Frontend Interaction (WebSockets & Streaming):** Chat messages, voting counts, and itinerary updates sync live via WebSockets. For detailed suggestions, Server-Sent Events (SSE) stream text smoothly onto the screen.
2. **Backend Gateway (FastAPI & Pydantic v2):** FastAPI receives requests, checks user login (JWT), validates data with Pydantic v2, and runs the DeepSeek-V4 agent in the background without freezing the server.
3. **Database Storage for User & Trip Data (PostgreSQL 16):**
 - **User Inputs Saved to DB:** Chat messages, budget limits, dietary needs, and votes are safely stored in PostgreSQL tables (`chat_messages`, `event_members`, `votes`).
 - **Session Checkpoints:** LangGraph's `AsyncPostgresSaver` saves conversation memory and plan drafts under `event_id`.
 - **Relational Itineraries:** Saved in normalized tables (`events` → `plans` → `plan_days` → `plan_stops`).

- **Live Atomic Updates:** Each venue is stored as its own row in `plan_stops`, so the agent can quickly update a single stop without re-creating the entire plan.
4. External Research APIs & Local Knowledge:
- **Hotels & Homestays (Agoda, Booking.com, Airbnb):** Live pricing, photos, and room availability.
 - **Dining & Cafes (Google Places, TripAdvisor):** Real opening hours, ratings, and exact map coordinates.
 - **Tours & Experiences (Klook, Traveloka):** Activity tickets and guided excursions with clear rates.
 - **Transit & Weather (Google Flights, 12Go Asia, OpenWeatherMap):** Travel times, schedules, and rain forecasts.
 - **Local Knowledge (pgvector RAG):** Semantic vector search for authentic local hidden gems.
 - **Redis Fast Cache:** Caches frequent search results (2h–24h TTL) to respond in under 50ms and reduce API costs.

3.3. Group Planning Workflow: Activity Diagram & Concrete Example

Figure 3 outlines the workflow for a 4-person Da Nang trip across three swimlanes:

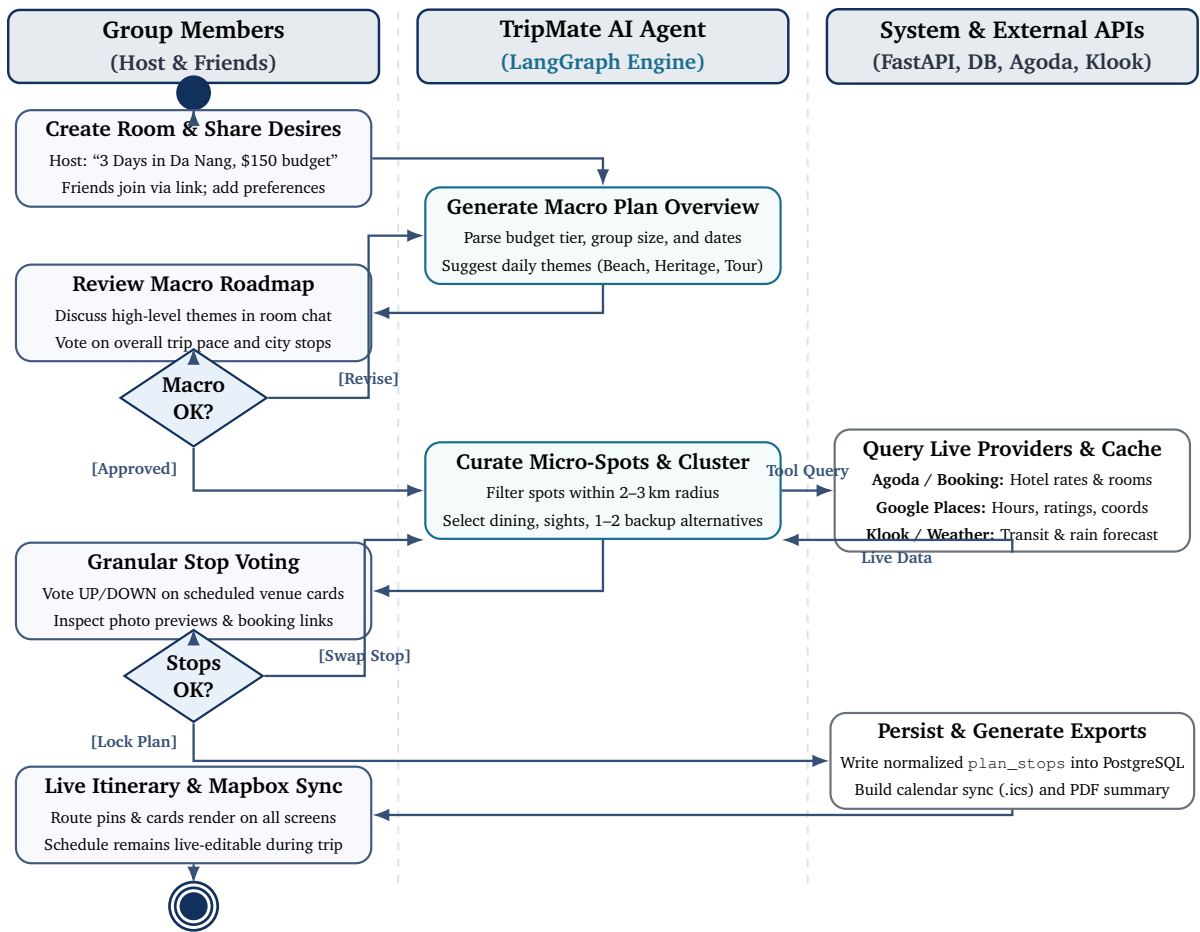


Figure 3: UML activity diagram: end-to-end group planning workflow with concrete 3-day trip example across user, AI agent, and API partitions

The planning session moves through four clear stages:

- **Intake & Preference Capture:** The Host initiates the room with core criteria (e.g., "3 Days in Da Nang, \$150 budget"). Participants join via link and add food or activity preferences.
- **Macro Roadmap Consensus:** The AI proposes daily themes (Day 1: Beach & Seafood, Day 2: Marble Mountains & Hoi An, Day 3: Han River & Cafes). Group members discuss and vote to lock this general roadmap.
- **Micro-Spot Curation & Granular Voting:** The AI calls external travel APIs to populate daily schedules with concrete venues clustered within a 2–3 km radius per half-day block. Members vote

- thumbs UP/DOWN on individual stops, and any downvoted spot offers a 1-click alternative.
- **Confirmation & Live Sync:** The Host locks the final plan. Normalized records are stored in PostgreSQL, calendar (.ics) and PDF summaries are generated, and interactive Mapbox routes appear on all devices.

4. System Architecture & Operational Flow

TripMate AI uses an asynchronous, decoupled architecture connecting the React SPA, FastAPI gateway, PostgreSQL storage, the unified LangGraph AI agent, and external travel APIs. Figure 4 presents the complete system architecture and highlights the six primary communication interfaces (1)–(6) that coordinate the platform.

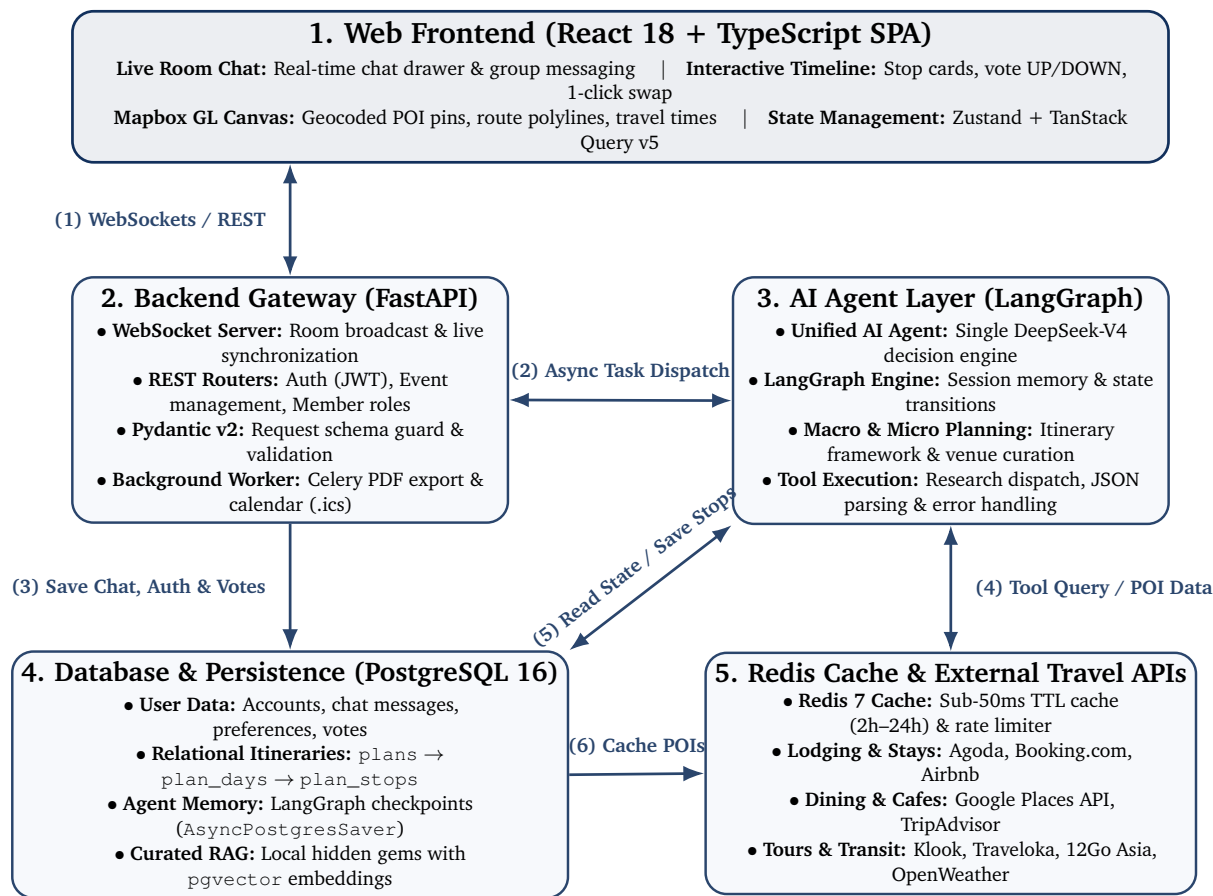


Figure 4: TripMate AI system architecture: component layers and operational interfaces (1)–(6)

4.1. End-to-End Operational Flow

Figure 5 visualizes the execution lifecycle of interfaces (1)–(6) from the System Architecture (Figure 4), explicitly distinguishing sequential dependencies from parallel processing tracks. Table 1 summarizes each operational stage, execution mode, and target latency.

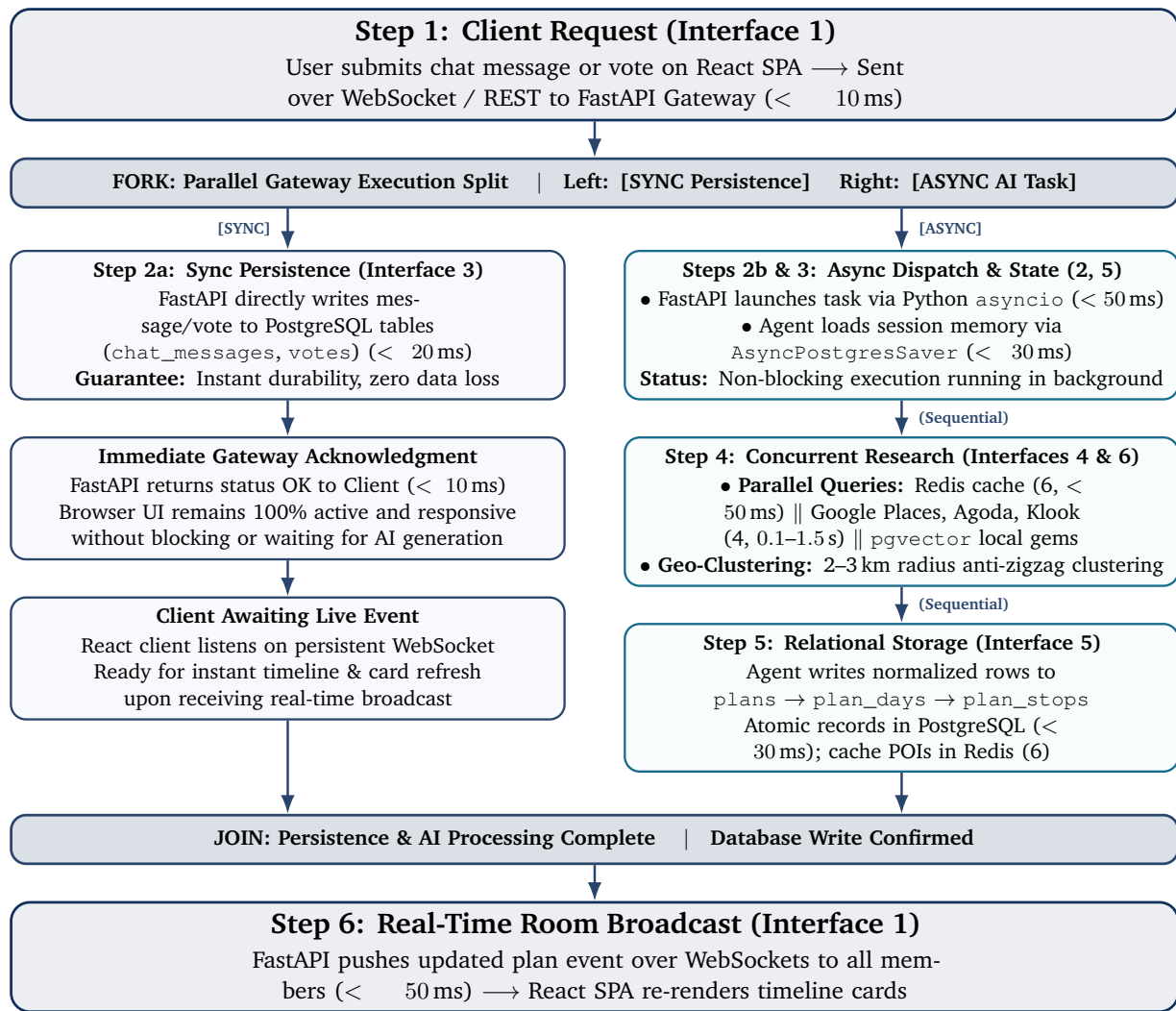


Figure 5: End-to-end operational execution flow: sequential progression, parallel fork/join tracks, and interface mappings (1)–(6)

Step	Interface Path	Mode	Depends On	Operational Function	Latency
1	(1) Frontend → Gateway	Sync	User action	Transmits message/vote via WebSocket or REST.	< 10 ms
2a	(3) Gateway → Database	Sync	Step 1	Persists chat logs and vote counts into PostgreSQL.	< 20 ms
2b	(2) Gateway → AI Agent	Async	Step 1 (parallel)	Dispatches non-blocking agent task via <code>asyncio</code> .	< 50 ms
3	(5) AI Agent ← Database	Async	Step 2b	Loads session state via <code>AsyncPostgresSaver</code> .	< 30 ms
4	(4, 6) AI ↔ APIs / Redis	Parallel	Step 3	Checks Redis cache; queries Places, Agoda, Klook concurrently.	0.1–1.5 s
5	(5) AI Agent → Database	Async	Step 4	Writes normalized records to <code>plan_stops</code> .	< 30 ms
6	(1) Gateway → Frontend	Push	Step 5	Broadcasts updated itinerary to all members via WebSocket.	< 50 ms

Table 1: Operational execution flow mapped directly to Figure 4 interfaces

Dynamic On-the-Go Adaptability: If travel conditions change unexpectedly (e.g., sudden rain), a member sends a simple message such as “*it is raining, find an indoor cafe nearby*”. The system re-executes this exact asynchronous loop: FastAPI logs the request (Step 2a), the agent queries nearby

cafes in parallel (Step 4), modifies **only the affected single row in plan_stops** in PostgreSQL (Step 5), and pushes the updated pin to all members in under two seconds (Step 6).

4.2. Technology Stack

The technical stack is chosen for speed, reliability, and smooth real-time collaboration:

Tier	Technology	Function & Purpose
Frontend	React 18 + TypeScript	Reliable Single Page Application for smooth UI rendering.
	Vite	Fast development server and optimized production build.
	Tailwind CSS + shadcn/ui	Clean, mobile-friendly design components.
	TanStack Query v5	Fast server caching and instant UI feedback on votes.
	Mapbox GL JS	Interactive maps, custom pins, and route navigation.
Backend	FastAPI (Python 3.11+)	High-speed async REST endpoints and automatic Swagger docs.
	WebSockets / SSE	Live group room chat and streaming AI responses.
	SQLAlchemy 2.0 (Async)	Non-blocking database access with Alembic migrations.
	Pydantic v2	Fast data validation and schema protection.
AI & Storage	LangGraph Python	Multi-step agent orchestration and state-machine flow.
	DeepSeek-V4	Single unified AI model for chat, planning, clustering, and tool calling.
	PostgreSQL 16 + pgvector	Relational database for events and plans; vector search for local spots.
	Redis 7	Fast API cache (<50ms), session store, and rate limiter.

Table 2: Complete technology stack for TripMate AI

4.3. Deployment, Security & Rate Limiting

- **Docker Containers:** Docker Compose packages backend services, Redis, and PostgreSQL for simple local testing and cloud deployment.
- **Cloud Hosting:** Frontend hosted on Vercel edge CDN; Backend hosted on Render / Linux VPS.
- **Security:** Passwords hashed with `bcrypt`; stateless JWT access tokens paired with Redis-managed refresh tokens; Room roles (Host/Member/Viewer).
- **Cost Protection:** Redis rate limiters prevent excessive AI token usage or runaway external API calls per room.

5. Expected Impact & Conclusion

TripMate AI solves the common headaches of group planning:

- **Under 15 Minutes:** Replaces days of arguing in group chats with a fast, guided 15-minute agreement session.
- **No Stress on One Person:** Group voting and instant alternatives ensure everyone has a voice without one organizer doing all the exhausting work.
- **Accurate & Grounded:** Recommendations come from verified sources (Agoda, Booking, Google Places, Klook), preventing closed shops or unrealistic schedules.
- **Live & Flexible:** Because the plan lives in an active database, groups can modify their schedule anytime, before or during the trip.

TripMate AI turns group trip planning from a headache into a fun, effortless, and collaborative experience.